Formal Verification Report: eBTC Protocol

- Repository: https://github.com/alexzoid-eth/2023-10-badger-fv
- Date: Nov 2023
- Author: @alexzoid (Code4rena formal verification contest)
- Certora Prover version: 4.13.1

Table of Contents

1. About eBTC Protocol
2. Formal Verification Methodology
    - Types of Properties
    - Verification Process
    - Project Structure
    - Assumptions
3. Verification Properties
    - ActivePool
    - CollSurplusPool
    - EBTCToken
    - SortedCdps
4. Mutation Testing
    - What is Mutation Testing
    - Manual Mutations
    - Gambit-Generated Mutations
    - Testing Scripts
5. Setup and Execution
    - Common Setup
    - Remote Execution
    - Running Verification
    - Running Mutation Testing

About eBTC Protocol

eBTC is a decentralized Bitcoin-pegged stablecoin protocol built by Badger DAO. Users open Collateralized Debt Positions (CDPs) backed by stETH (Lido staked ETH) to mint eBTC tokens. The protocol maintains a sorted list of CDPs ordered by collateralization ratio to facilitate efficient liquidations and redemptions.

Four contracts are in scope for formal verification:

1. **ActivePool** (212 SLOC) — Manages the protocol's active collateral shares and system debt. Handles collateral token transfers between protocol components, ERC-3156 flash loans of collateral, fee recipient management, and debt accounting restricted to BorrowerOperations and CdpManager.
2. **CollSurplusPool** (83 SLOC) — Holds surplus collateral shares from liquidated CDPs. Manages per-user surplus balances, ensures surplus can only increase (except when claimed to zero), and allows users to claim their surplus collateral.
3. **EBTCToken** (201 SLOC) — The eBTC ERC-20 token with EIP-2612 permit support. Implements restricted minting and burning accessible only to authorized protocol contracts (BorrowerOperations, CdpManager).
4. **SortedCdps** (339 SLOC) — A sorted doubly-linked list of CDPs ordered by their Nominal Individual Collateral Ratio (NICR) in descending order. Supports insert, remove, batch remove, and re-insert operations for efficient CDP management.

Key operations:

- `transferSystemCollShares` / `transferSystemCollSharesAndLiquidatorReward` — Transfer collateral shares out of the ActivePool
- `allocateSystemCollSharesToFeeRecipient` — Allocate fee shares from system collateral
- `flashLoan` — ERC-3156 flash loan of collateral tokens with configurable fee
- `increaseSurplusCollShares` / `claimSurplusCollShares` — Manage and claim surplus collateral
- `mint` / `burn` / `transfer` — ERC-20 token operations with caller restrictions
- `insert` / `remove` / `reInsert` / `batchRemove` — Sorted linked list operations maintaining NICR ordering

Formal Verification Methodology

Certora Formal Verification (FV) provides mathematical proofs of smart contract correctness by verifying code against a formal specification. Unlike testing and fuzzing which examine specific execution paths, Certora FV examines all possible states and execution paths.

The process involves crafting properties in CVL (Certora Verification Language) and submitting them alongside compiled Solidity smart contracts to a remote prover. The prover transforms the contract bytecode and rules into a mathematical model and determines the validity of rules.

Types of Properties

Properties are categorized following the [official Certora methodology](). Valid State, Variable Transitions and State Transitions properties are **parametric** — they are automatically verified against every external function in the contract, including functions added after the specification is completed. High-Level properties target specific function sequences.

Valid State — System-wide invariants that MUST always hold true. These properties define the fundamental constraints of the protocol, such as accounting consistency and structural integrity. Once proven, these invariants serve as trusted assumptions in other properties via `requireInvariant`, reducing verification complexity.

Variable Transitions — Properties that verify specific storage variables change only under expected conditions. The process captures a variable value before instruction execution, runs the instruction, then asserts the variable changed only as permitted or remained unchanged.

State Transitions — Properties that verify the correctness of transitions between valid states. Building upon the valid state invariants, these properties ensure the protocol's state machine operates correctly and that state changes are both authorized and sequentially valid.

High-Level — Complex multi-step properties verifying business logic integrity. Unlike parametric properties, these target specific function sequences to validate end-to-end protocol behavior.

Unit Tests — Properties that verify basic behavior of individual functions: revert conditions, direct effects on state, and non-effects on unrelated state. Unlike parametric properties, each unit test targets a specific function call.

Verification Process

1. **Setup phase**: Define ghost variables, storage hooks, and helper definitions to model contract state in CVL. Establish verification harnesses and configurations for four independent targets (ActivePool, CollSurplusPool, EBTCToken, SortedCdps). The setup is organized into base specs and dependency specs:
    - Base specifications model each contract's internal state:
        - `base/activePool.spec`: Ghost variables for `feeRecipientAddress`, `systemCollShares`, `systemDebt`, and `feeRecipientCollShares` with storage hooks tracking current and previous values, plus change-detection flags
        - `base/collateralTokenTester.spec`: Ghost-based ERC-20 collateral token model tracking `_totalBalance` and per-address `balances` with sum accumulator for solvency invariants

        - `base/collSurplusPool.spec`: Ghost variables for `totalSurplusCollShares` and per-user `balances` with change-detection flags and user-address tracking
        - `base/eBTCToken.spec`: Ghost variables for `_totalSupply`, per-address `_balances` with sum tracking, and `_allowances` mapping for ERC-20 compliance verification
        - `base/sortedCdps.spec`: Comprehensive linked list ghost structure modeling `head`, `tail`, `size`, `nextId`/`prevId` mappings, `nextCdpNonce`, `cachedNominalICR`, and a `uniqueId` axiom for CDP ID generation
    - Dependency specifications model external contract interfaces:
        - `dependencies/AuthNoOwner.spec`: Ghost variables for `_authority` address and `_authorityInitialized` flag, with invariants ensuring proper initialization
        - `dependencies/ReentrancyGuard.spec`: Ghost variable for reentrancy `locked` state (OPEN=1, LOCKED=2)
        - `dependencies/ERC3156FlashLender.spec`: Ghost variables for `feeBps` and `flashLoansPaused`, with invariant bounding fees to `MAX_FEE_BPS`
        - `dependencies/CdpManager.spec`: Method declarations for `authority()`, `syncGlobalAccountingAndGracePeriod()`, and `getCdpStatus()`; ghost variables for CDP status mapping; definitions for CDP status constants
        - `dependencies/ERC20.spec`: Standard ERC-20 method declarations
        - `dependencies/PermitNonce.spec`: Ghost variables for `_nonces` tracking with monotonicity invariant
        - `dependencies/helper.spec`: Helper sanity rule and `VIEW_PURE_FUNCTIONS` definition for method filtering
2. **Crafting Properties**: Write invariants and rules in CVL, starting with valid state invariants (which become trusted preconditions for other rules), then state transitions, and finally high-level and unit test properties. Properties are organized per-contract in the main spec files, importing required base and dependency specs.

Project Structure

```
certora/
├── confs/                                  # Prover configuration files
│   ├── ActivePool_verified.conf            # ActivePool verification config
│   ├── CollSurplusPool_verified.conf       # CollSurplusPool verification config
│   ├── EBTCToken_verified.conf             # EBTCToken verification config
│   └── SortedCdps_verified.conf            # SortedCdps verification config
│
├── harness/                                # Verification harnesses
│   ├── ActivePoolHarness.sol               # ActivePool harness
│   ├── CollSurplusPoolHarness.sol          # CollSurplusPool harness with
call_isAuthorized
│   ├── EBTCTokenHarness.sol                # EBTCToken harness
│   ├── SortedCdpsHarness.sol               # SortedCdps harness with list helpers
│   ├── CollateralTokenTester.sol           # Collateral ERC-20 mock (stETH model)
│   ├── DummyERC20Impl.sol                  # Dummy ERC-20 implementation
│   ├── DummyERC20A.sol                     # Dummy ERC-20 instance A
│   └── DummyERC20B.sol                     # Dummy ERC-20 instance B
```

```
│
├── mutations/                                # Mutation testing files
│   ├── ActivePool/                           # Manual mutations (5 files)
│   ├── CollSurplusPool/                      # Manual mutations (5 files)
│   ├── EBTCToken/                            # Manual mutations (8 files)
│   ├── SortedCdps/                           # Manual mutations (4 files)
│   ├── ActivePool_gambit174/                 # Gambit-generated mutations (173 files)
│   ├── CollSurplusPool_gambit58/             # Gambit-generated mutations (58 files)
│   ├── EBTCToken_gambit147/                  # Gambit-generated mutations (147 files)
│   ├── SortedCdps_gambit292/                 # Gambit-generated mutations (275 files)
│   ├── ActivePool_results/                   # Mutation testing results
│   ├── CollSurplusPool_results/              # Mutation testing results
│   ├── EBTCToken_results/                    # Mutation testing results
│   ├── SortedCdps_results/                   # Mutation testing results
│   ├── EBTCToken_fix/                        # EBTCToken fix mutations
│   ├── SortedCdps_fix/                       # SortedCdps fix mutations
│   ├── gambitToMutations.sh                  # Copy Gambit mutations to certora directory
│   ├── addMutation.sh                        # Create new manual mutations with diff
comments
│   ├── checkMutation.sh                      # Apply mutations, run prover, restore files
│   └── checkMutationsStatus.py               # Check mutation testing status
│
└── specs/                                    # CVL specifications
    ├── ActivePool.spec                       # ActivePool properties (27 rules/invariants)
    ├── CollSurplusPool.spec                  # CollSurplusPool properties (13
rules/invariants)
    ├── EBTCToken.spec                        # EBTCToken properties (9 rules/invariants)
    ├── SortedCdps.spec                       # SortedCdps properties (28 rules/invariants)
    ├── base/                                 # Base spec files (ghost variables, hooks)
    │   ├── activePool.spec                   # ActivePool state ghosts and hooks
    │   ├── collateralTokenTester.spec        # Collateral ERC-20 model
    │   ├── collSurplusPool.spec              # CollSurplusPool state ghosts and hooks
    │   ├── eBTCToken.spec                    # EBTCToken balance/supply model
    │   └── sortedCdps.spec                   # Linked list ghost structure
    └── dependencies/                         # External contract models
        ├── AuthNoOwner.spec                  # Authority initialization invariants
        ├── CdpManager.spec                   # CdpManager interface and CDP status
        ├── ERC20.spec                        # Standard ERC-20 interface
        ├── ERC3156FlashLender.spec           # Flash lender fee constraints
        ├── helper.spec                       # Sanity check rule
        ├── PermitNonce.spec                  # Permit nonce monotonicity
        └── ReentrancyGuard.spec              # Reentrancy guard state
```

Assumptions

Formal verification requires assumptions about the code and its environment to address prover timeouts, tool limitations, and state consistency. However, incorrect assumptions can mask real bugs by excluding reachable states from analysis. To maintain transparency, all assumptions are categorized into three groups: **Safe** (real-world constraints that don't reduce security coverage), **Proved** (formally verified invariants reused as preconditions), and **Unsafe** (scope reductions necessary for tractability that may exclude valid scenarios).

Safe Assumptions

These reflect real-world constraints that don't impact security guarantees.

Collateral Token Solvency (`base/collateralTokenTester.spec`):

- The collateral token's total balance equals the sum of all individual balances (standard ERC-20 solvency invariant)
- Individual collateral balances are bounded below `max_uint128` to prevent solver overflow

Address Separation (various specs):

- Fee recipient address is distinct from the ActivePool contract and CollSurplusPool
- Flash loan receiver is distinct from fee recipient and the ActivePool contract
- User addresses in CollSurplusPool are non-zero (enforced by the Sstore hook)

Ghost Variable Initialization (all base specs):

- All ghost variables are initialized to zero/false via `init_state axiom`
- Change-detection ghost flags (`ghostSystemCollSharesChanged`, `ghostBalancesChanged`, etc.) start as `false` and are reset in `preserved` blocks

Proved Assumptions

These properties have been formally verified as valid state invariants and are used as trusted preconditions (via `requireInvariant`) in other rules.

ActivePool Proved Invariants:

- `collateralBalancesSolvency` — Collateral total balance equals sum of all balances (`base/collateralTokenTester.spec`), used in `flashLoanIntegrity`

CollSurplusPool Proved Invariants:

- `decreaseTotalSharesUserSharesSolvency` and `decreaseUserSharesTotalSharesSolvency` — Used in `decreaseUserBalanceTransferTokens` to establish that total shares and user balance changes are atomic

SortedCdps Proved Invariants (all defined in `base/sortedCdps.spec`):

- `isListStructureValidInv` — List structure is valid (head/tail/next/prev consistency)
- `sortedCdpsMaxSizeGtZero` — Max size is non-zero
- `sortedCdpsSizeLeqMaxSize` — Current size does not exceed max size
- `sortedCdpsListEmptySolvency` — Empty list has zeroed head/tail/links
- `sortedCdpsListOneItemSolvency` — Single-item list has consistent head=tail with no links
- `sortedCdpsListSeveralItemsSolvency` — Multi-item list has correct bidirectional linking
- `sortedCdpsListNotLinkedToSelf` — No node links to itself
- `sortedCdpsListZeroIdsNotSupported` — Zero-id nodes cannot have links
- `sortedCdpsListNoDuplicates` — No duplicate prev/next pointers
- `sortedCdpsListNICRDescendingOrder` — NICR values follow descending order

These invariants are collectively applied via the `setupList()` function in the SortedCdps base spec, which calls `requireInvariant` on all ten list structure invariants before any rule that operates on the list.

Unsafe Assumptions

These reduce verification scope to make the problem tractable for the prover.

Optimistic Loop Unrolling (all configurations):

- `optimistic_loop: true` — The prover assumes loops terminate within the unrolled bound. Any behavior requiring more iterations than the bound is excluded from verification.
- Default loop iteration bound of 1 for most contracts; `loop_iter: 4` for SortedCdps

Bounded List Size (`base/sortedCdps.spec`):

- `maxSize() <= 4` required in `setupList()` — Bounds the sorted linked list to at most 4 elements for tractable verification of universal quantifiers over list nodes. Real deployments may use much larger lists.

External Call Summaries:

- `MockFlashBorrower.onFlashLoan()` is summarized as a CVL function (`onFlashLoanCVL`) that captures parameters and returns a configurable ghost value. The actual borrower callback behavior is not modeled.
- `CdpManager.getCachedNominalICR()` is summarized via a ghost mapping (`cachedNominalICR`). Real ICR computation logic is not verified.
- `SortedCdps.toCdpId()` internal function is summarized with a `uniqueId` ghost axiom ensuring uniqueness. The actual keccak256-based ID generation is replaced.

Verification Properties

Links to specific CVL spec files are provided for each property, with status indicators.

- ✅ Verified

Total properties: 77 (27 ActivePool + 13 CollSurplusPool + 9 EBTCToken + 28 SortedCdps)

ActivePool

The ActivePool specification verifies collateral share management, system debt accounting, flash loan integrity, fee management, and access control. Properties are defined in `ActivePool.spec` with ghost variables and hooks from `base/activePool.spec`, `base/collateralTokenTester.spec`, and `base/collSurplusPool.spec`.

Property	Name	Description	Status	Notes
AP-01	`helperSanity`	Sanity check — possibility of not being reverted	✅	Defined in `dependencies/helper.spec`
AP-02	`authNoOwnerInitializedAndAddressSetInConstructor`	[2] In initialized state authority address should not be zero	✅	Defined in `dependencies/AuthNoOwner.spec`
AP-03	`authNoOwnerSetAuthorityFromCdpManagerInConstructor`	[g4] authority() should be set in constructor from CdpManager	✅	Defined in `dependencies/AuthNoOwner.spec`
AP-04	`erc3156FlashLenderFeeBpsNotGtMaxfeeBps`	[3] feeBps should not be greater than MAX_FEE_BPS	✅	Defined in `dependencies/ERC3156FlashLender.spec`
AP-05	`activePoolIncreaseCollSurplusPoolTotalSharesShouldTransferCollateralTokens`	[4] When CollSurplusPool total shares increased, collateral tokens should be transferred	✅	Defined in `base/activePool.spec`
AP-06	`onlyBorrowerOperationsIncreaseSystemCollShares`	Only BorrowerOperations should increase systemCollShares	✅	
AP-07	`onlyCdpManagerAllocateFeeFromShares`	Only CdpManager should increase fees	✅	
AP-08	`feeRecipientCollSharesIncreaseSolvency`	Fees should be taken from shares — fee increase equals shares decrease	✅	
AP-09	`modifyStateCallerRestrictions`	[5] Only CdpManager, BorrowerOperations or authorized user should modify state	✅	
AP-10	`oneWayChangeSystemDebt`	SystemDebt should be changed by BorrowerOperations or CdpManager with increaseSystemDebt() or decreaseSystemDebt()	✅	
AP-11	`syncedGlobal`	[g150] syncGlobalAccountingAndGracePeriod() should be called for fee/config functions	✅	Filtered to specific functions
AP-12	`transferSystemCollSharesDecreaseTotalShares`	[g10,g11,g13,g14,g16] Transfer collateral shares decrease total shares by the transferred amount	✅	
AP-13	`transferToCollSurplusPoolIncreaseTotalSurplusCollShares`	[g54-56] Transfer to CollSurplusPool should increase its total shares and token balance atomically	✅	
AP-14	`transferSystemCollSharesAndLiquidatorRewardDecreaseTotalShares`	[g22-28,g30-33] Transfer collateral shares and liquidator reward decrease total shares	✅	
AP-15	`allocateSystemCollSharesToFeeRecipientDecreaseTotalShares`	[g45-51] Fee allocation decreases system shares and increases fee shares by equal amounts	✅	
AP-16	`changeSystemDebtIntegrity`	[g69,g70,g73,g74] increaseSystemDebt increases and decreaseSystemDebt decreases by exact amount	✅	
AP-17	`increaseSystemCollSharesIntegrity`	[g84-91] increaseSystemCollShares increases shares by exact amount	✅	
AP-18	`collateralBalancesSolvency`	Collateral total balance equals sum of all individual balances	✅	Defined in `base/collateralTokenTester.spec`
AP-19	`flashLoanIntegrity`	[g92-93,g96-97,g99-106,g108-110,g112-113,g115-116] flashLoan() validates amount, callback parameters, fee transfer, balance solvency, and rate invariance	✅	
AP-20	`flashFeeIntegrity`	[g118-119,g121-122,g124-134] flashFee() returns correct fee as (amount * feeBps) / MAX_BPS	✅	
AP-21	`maxFlashLoanIntegrity`	[g136,138] maxFlashLoan() returns collateral balance for valid token, zero otherwise	✅	
AP-22	`claimFeeRecipientCollSharesIntegrity`	[g143-144,g146,g149] claimFeeRecipientCollShares() decreases fee shares, contract balance, and increases recipient balance by equal amounts	✅	
AP-23	`sweepTokenNotCollateral`	[g151-152] Collateral token cannot be swept	✅	
AP-24	`sweepTokenIntegrity`	[g154-155,g157-158] sweepToken() transfers tokens from contract to fee recipient	✅	
AP-25	`setFeeRecipientAddressIntegrity`	[g159-160,g162] setFeeRecipientAddress() sets non-zero address correctly	✅	
AP-26	`setFeeBpsIntegrity`	[g168-170] setFeeBps() stores the new fee value	✅	
AP-27	`setFlashLoansPausedIntegrity`	[g172-174] setFlashLoansPaused() stores the new paused state	✅	

CollSurplusPool

The CollSurplusPool specification verifies surplus collateral management, user balance integrity, and access control. Properties are defined in `CollSurplusPool.spec` with ghost variables from `base/collSurplusPool.spec`.

Property	Name	Description	Status	Notes
CSP-01	`helperSanity`	Sanity check — possibility of not being reverted	✅	Defined in `dependencies/helper.spec`
CSP-02	`authNoOwnerInitializedAndAddressSetInConstructor`	In initialized state authority address should not be zero	✅	Defined in `dependencies/AuthNoOwner.spec`
CSP-03	`authNoOwnerSetAuthorityFromCdpManagerInConstructor`	authority() should be set in constructor from CdpManager	✅	Defined in `dependencies/AuthNoOwner.spec`
CSP-04	`changeToCollateralBalance`	Collateral balance decrease requires authorized caller or BorrowerOperations	✅	
CSP-05	`userBalanceShouldAlwaysIncreaseExceptSetToZero`	[2] User balances should always increase except when set to zero	✅	
CSP-06	`decreaseUserSharesTotalSharesSolvency`	[3] User shares can decrease only to zero and must decrease total shares simultaneously by previous user balance	✅	
CSP-07	`decreaseTotalSharesUserSharesSolvency`	[4] Total shares decrease requires exactly one user's balance to be zeroed, decrease equals previous user balance	✅	
CSP-08	`decreaseUserBalanceTransferTokens`	[5] Decrease user balance should transfer tokens from contract to user	✅	
CSP-09	`increaseSurplusCollSharesIntegrity`	increaseSurplusCollShares() only callable by CdpManager, increases user balance by exact amount	✅	
CSP-10	`claimSurplusCollSharesIntegrity`	claimSurplusCollShares() decreases total shares by user balance, requires non-zero balance, zeroes user balance	✅	
CSP-11	`increaseTotalSurplusCollSharesIntegrity`	increaseTotalSurplusCollShares() only callable by ActivePool, increases total shares by exact amount	✅	
CSP-12	`sweepTokenNotCollateral`	Collateral token cannot be swept	✅	
CSP-13	`sweepTokenIntegrity`	sweepToken() transfers tokens from contract to fee recipient	✅	

EBTCToken

The EBTCToken specification verifies ERC-20 token integrity, supply solvency, balance constraints, and permit nonce management. Properties are defined in `EBTCToken.spec` with ghost variables from `base/eBTCToken.spec`.

Property	Name	Description	Status	Notes
ET-01	`helperSanity`	Sanity check — possibility of not being reverted	✅	Defined in `dependencies/helper.spec`
ET-02	`mingIntegrity`	Mint integrity — account balance increases by exact amount, other users unaffected	✅	
ET-03	`permitNonceAlwaysIncremented`	[1] Nonce always incremented	✅	Defined in `dependencies/PermitNonce.spec`
ET-04	`eBTCTokenBalanceOfMax2UsersChanged`	[2] Balance of max two users could be changed at a time	✅	Defined in `base/eBTCToken.spec`
ET-05	`eBTCTokenTotalSupplySolvency`	[3] Total supply storage variable equals sum of all balances	✅	Defined in `base/eBTCToken.spec`
ET-06	`eBTCTokenSumOfAllBalancesEqTotalSupply`	[4] Total supply equals sum of tracked user balances	✅	Defined in `base/eBTCToken.spec`
ET-07	`eBTCTokenBalanceOfAddressZeroNotChanged`	[5] Balance at address(0) should not change	✅	Defined in `base/eBTCToken.spec`

Property	Name	Description	Status	Notes
ET-08	`eBTCTokenTransferToRecipientRestrictions`	[6] Recipient cannot be address(0) or the token contract itself	✅	Defined in `base/eBTCToken.spec`
ET-09	`eBTCTokenAllowanceOfAddressZeroNotChanged`	[7-8] Allowance at address(0) should not change	✅	Defined in `base/eBTCToken.spec`

SortedCdps

The SortedCdps specification verifies sorted linked list integrity, access control, NICR ordering, and operation correctness. Properties are defined in `SortedCdps.spec` with the comprehensive linked list ghost model from `base/sortedCdps.spec`.

Property	Name	Description	Status	Notes
SC-01	`helperSanity`	Sanity check — possibility of not being reverted	✅	Defined in `dependencies/helper.spec`
SC-02	`uniqunessOfId`	toCdpId() returns unique result for unique input parameters	✅	
SC-03	`changeToFirst`	List head can only change via insert, reInsert, remove, or batchRemove	✅	
SC-04	`isFullCanNotIncrease`	When list is full, size cannot increase	✅	
SC-05	`isListStructureValidInv`	List structure is valid (harness function check)	✅	Defined in `base/sortedCdps.spec`
SC-06	`sortedCdpsMaxSizeGtZero`	[2] Max size cannot be zero	✅	Defined in `base/sortedCdps.spec`
SC-07	`sortedCdpsSizeLeqMaxSize`	[3] Current size does not exceed max size	✅	Defined in `base/sortedCdps.spec`
SC-08	`sortedCdpsListEmptySolvency`	Empty list solvency — if any of head/tail/size is zero, all must be zero with no links	✅	Defined in `base/sortedCdps.spec`
SC-09	`sortedCdpsListOneItemSolvency`	Single-item list solvency — head equals tail, size is 1, non-zero NICR, no prev/next links	✅	Defined in `base/sortedCdps.spec`
SC-10	`sortedCdpsListSeveralItemsSolvency`	[4] Multi-item list solvency — correct bidirectional linking, non-zero NICRs, proper head/tail structure	✅	Defined in `base/sortedCdps.spec`
SC-11	`sortedCdpsListNotLinkedToSelf`	Items in the list cannot link to themselves	✅	Defined in `base/sortedCdps.spec`
SC-12	`sortedCdpsListZeroIdsNotSupported`	Items with zero ID cannot have prev/next links	✅	Defined in `base/sortedCdps.spec`
SC-13	`sortedCdpsListNoDuplicates`	List does not contain duplicate prev/next pointers	✅	Defined in `base/sortedCdps.spec`
SC-14	`sortedCdpsListNICRDescendingOrder`	SL-01 NICR ranking follows descending order in the sorted list	✅	Defined in `base/sortedCdps.spec`
SC-15	`onlyBOorCdpMShouldModityState`	Only BorrowerOperations or CdpManager should modify contract state	✅	
SC-16	`onlyInsertShouldIncreaseSizeAndNonce`	Only insert should increase list size and next CDP nonce	✅	
SC-17	`nextCdpNonceIncrements`	Next CDP nonce always increments by exactly 1	✅	

Property	Name	Description	Status	Notes
SC-18	`reInsertNotAffectSize`	reInsert() does not change list size	✅	
SC-19	`reInsertRemoveExistingId`	reInsert() reverts if the ID is not in the list	✅	
SC-20	`onlyRemoveFunctionsShouldIncreaseSize`	Only remove and batchRemove should decrease list size	✅	
SC-21	`onlyCdpMShouldRemove`	Only CdpManager should remove IDs from the list	✅	
SC-22	`containsIntegrity`	[g239-243] contains() returns true iff node is in list	✅	
SC-23	`getOwnerAddressIntegrity`	[g14-20] getOwnerAddress() extracts the correct address from CdpId	✅	
SC-24	`removeWhenMaxSize`	[g228] Possibility to remove() when list is at maximum size	✅	
SC-25	`validInsertPositionIntegrity`	[g245] validInsertPosition() correctly validates position based on NICR ordering	✅	
SC-26	`maxSizeNotZeroInConstructor`	maxSize() cannot be set to zero in constructor	✅	
SC-27	`batchRemoveDecreaseSize`	[g189] batchRemove() decreases size by exactly the number of removed IDs	✅	
SC-28	`batchRemoveOnlyExistingIdsSupported`	[g160-163] batchRemove() reverts if any ID is not in the list	✅	

Mutation Testing

What is Mutation Testing

Mutation testing validates the strength of a formal specification by injecting small syntactic changes (mutations) into the source code and verifying that the spec catches them. Each mutation represents a potential bug — a removed assignment, a flipped operator, an off-by-one error.

The certoraMutate tool automates this process: it applies each mutant to the source, runs the prover against the specified rules, and reports whether the specification caught the introduced defect.

This project uses two complementary mutation approaches:

Manual Mutations

Manual mutations are hand-crafted code changes targeting specific contract behaviors. These are stored in per-contract directories and referenced by the `manual_mutants` section in each verification configuration file.

Contract	Mutations	Location
ActivePool	5	`mutations/ActivePool/`
CollSurplusPool	5	`mutations/CollSurplusPool/`
EBTCToken	8	`mutations/EBTCToken/`
SortedCdps	4	`mutations/SortedCdps/`
Total	**22**	

Manual mutations serve as the publicly available participation mutations for the Code4rena FV contest. Catching all public mutations qualifies for the participation reward.

Gambit-Generated Mutations

Gambit automatically generates mutations by applying systematic syntactic transformations to the Solidity source code. These mutations provide broader coverage than manual mutations.

Contract	Mutations	Location
ActivePool	173	`mutations/ActivePool_gambit174/`
CollSurplusPool	58	`mutations/CollSurplusPool_gambit58/`
EBTCToken	147	`mutations/EBTCToken_gambit147/`
SortedCdps	275	`mutations/SortedCdps_gambit292/`
Total	**653**	

Results are stored in per-contract `_results/` directories. Fix mutations for identified issues are available in `EBTCToken_fix/` and `SortedCdps_fix/` directories.

Testing Scripts

Three shell scripts automate mutation testing workflows:

- `gambitToMutations.sh` — Copies Gambit-generated mutations from `gambit_out/mutants/` to the certora mutations directory with sequential numbering
- `addMutation.sh` — Creates new manual mutations with embedded git diff comments for traceability
- `checkMutation.sh` — Applies a mutation, runs the Certora Prover against the specification, and restores the original source file. Supports running full configs, specific mutations, or specific rules

Setup and Execution

The Certora Prover runs remotely using Certora's cloud infrastructure. The instructions below are for setting up the verification environment.

Common Setup

1. Install Java (JDK >= 17)

```
sudo apt update
sudo apt install default-jre
java -version
```

2. Install pipx — installs Python CLI tools in isolated environments

```
sudo apt install pipx
pipx ensurepath
```

3. Install Certora CLI (version 4.13.1)

```
pipx install certora-cli==4.13.1
```

4. Install solc-select and the required Solidity compiler

```
pipx install solc-select
solc-select install 0.8.17
solc-select use 0.8.17
```

Remote Execution

5. Set up Certora key. You can get a free key through the Certora Discord or on their website. Once you have it, export it:

```
echo "export CERTORAKEY=<your_certora_api_key>" >> ~/.bashrc
```

Running Verification

Run the verification for each contract using the `_verified` configuration files:

```
# ActivePool
certoraRun certora/confs/ActivePool_verified.conf

# CollSurplusPool
certoraRun certora/confs/CollSurplusPool_verified.conf

# EBTCToken
certoraRun certora/confs/EBTCToken_verified.conf

# SortedCdps
certoraRun certora/confs/SortedCdps_verified.conf
```

Running Mutation Testing

Run mutation testing using `certoraMutate` with the verified configuration:

```
# Example: Run ActivePool mutation testing
certoraMutate --prover_conf certora/confs/ActivePool_verified.conf \
    --mutation_conf certora/confs/gambit/ActivePool.conf

# Example: Run specific mutation manually
cd certora/mutations
./checkMutation.sh ActivePool

# Run specific mutation number
./checkMutation.sh ActivePool ../../../packages/contracts/contracts/ActivePool.sol 3

# Run specific rule against mutations
./checkMutation.sh ActivePool --rule flashLoanIntegrity
```